AI AGENT

PROMPT ENGINEERING GUIDE

Universal Template · Runtime Commands · Full Examples

Part I — Template Skeleton + Runtime Commands

Part II — Section Reference (for you, the human)

Part III — Full Prompt Examples

How to read this document: Part I contains everything that goes **into your prompt** — the skeleton and the runtime commands, both ready to copy. Part II explains each skeleton section **for you**. Part III shows five complete filled-in examples.

The runtime commands column (left) is written for the agent. The explanation column (right) is for you — it never enters the prompt.

Part I — Template Skeleton & Runtime Commands

This is what goes into your prompt. Fill in every [bracketed] field for your task. The Runtime Commands section is already written — paste it in as-is.

THE PROMPT SKELETON

```
# ROLE
You are [specific role, expertise, seniority, key traits].

# CONTEXT
## Situation
[What already exists. What triggered this task. What has been tried before.]

## Domain & Constraints
[Tech stack, language version, frameworks, platform, hard boundaries.]

## Audience
[Who will use this output. Their level. Their expectations.]

# TASK
## Primary Objective
[One sentence, action verb first: write / build / refactor / analyze / plan...]

## Sub-tasks
1. [Step or component]
2. [Step or component]
3. [Step or component]

# CONSTRAINTS
## Must Do
- [Non-negotiable requirement]
- [Non-negotiable requirement]

## Must Not Do
- [Hard exclusion]
- [Hard exclusion]

## Quality Bar
[production-ready | peer-review level | internal draft | proof-of-concept]

# OUTPUT FORMAT
## Structure: [sections, headers, code blocks, tables, ordering]
## Style:    [formal/informal, active voice, technical depth]
## Length:   [~N words / ~N lines of code / N sections]

# EXAMPLES
## Good output looks like:
[Paste or describe a reference]
```

```
## Avoid this:
[Name the most predictable failure mode]

# VERIFICATION
Before delivering your output, verify each item:
- [ ] [Specific, testable check]
- [ ] [Specific, testable check]
- [ ] [Specific, testable check]

# RUNTIME INSTRUCTIONS
──────────────────────────────────────────────────────────────────

• PLAN FIRST — Before writing any code, produce PLAN.md with: requirements,
  chosen stack & architecture, component map, open questions. Do not write
  implementation code until PLAN.md is complete. Reference and update it
  as you implement.

• EXPLAIN BEFORE YOU CODE — Before any non-trivial function, write a 3-5
  sentence plain-English description: what it does, algorithm used and why,
  inputs/outputs/side effects, edge cases. This becomes the docstring.
  If writing it reveals a design flaw, fix it before writing code.

• LOG EVERY ASSUMPTION — When assuming anything not explicitly stated,
  log it immediately: ASSUMPTION [#N]: what / Reason: why /
  Impact if wrong: what breaks. Collect all in ASSUMPTIONS.md.
  Never assume silently.

• RUN → CAPTURE → FIX → REPEAT — After writing or modifying code, run it
  immediately. For each error: state the exact message, hypothesize root
  cause, apply fix, re-run and verify. Repeat until end-to-end clean.
  If the same error persists after 2 attempts: log as DEAD END,
  abandon the approach, try a fundamentally different strategy.

• LOG DEAD ENDS — When an approach fails twice, do not retry it. Log in
  DEAD_ENDS.md: approach / failed because / what was tried / next strategy.
  Check DEAD_ENDS.md before trying any new approach.

• REGRESSION CHECK AFTER EVERY FIX — After any fix, list what was working
  before, re-test each item, confirm explicitly: 'Fix applied. X, Y, Z
  still correct.' Never close a bug without a passing regression check.

• MILESTONE CHECKPOINT AFTER EACH WORKING STATE — When code reaches a
  working state, write MILESTONE_N.md: status / what works / what is
  pending / decisions since last milestone / new assumptions.
  Do not proceed until current milestone is confirmed working.

• SELF-REVIEW PASS AFTER EACH MAJOR SECTION — After each component,
  do a dedicated review pass (separate from generation). Check for: logic
  errors, unhandled edge cases, inconsistencies with previous sections,
  gaps between what was asked and what was produced. Fix before continuing.
  Final full-output review before delivery using the VERIFICATION checklist.

• CONTEXT CHECKPOINT IF APPROACHING TOKEN LIMIT — Before hitting context
  limit, write CHECKPOINT.md: task summary / completed / in progress /
  pending / decisions made / assumptions / known issues / next action.
  Read CHECKPOINT.md before resuming after any context reset.
```

```
• UPDATE DOCUMENTATION AS YOU GO — Whenever implementation affects
  documented behavior, update docs immediately. README.md when
  setup/usage/behavior changes. Docstrings when contracts change.
  PLAN.md when architecture changes. Never defer docs to the end.
```

RUNTIME COMMANDS

> *These commands follow the skeleton. Copy this entire block into your prompt — they are written for the agent, not for you. The right column below explains each one (for you only).*

COMMAND (paste into prompt — for the agent)	WHY IT'S THERE (for you, the human)

• PLAN FIRST
`Before writing any code, produce a PLAN.md file containing:`
`  - Requirements: what the system must do (numbered list)`
`  - Stack & Architecture: chosen technologies and structure`
`  - Component map: what each module/file is responsible for`
`  - Open questions: anything unclear that needs a decision`
`Do not write any implementation code until PLAN.md is complete.`
`Reference and update PLAN.md as you implement.`
	Forces upfront design thinking before any code is written. Catches architectural mistakes when they are cheapest to fix. Keeps the agent oriented throughout the task.

• EXPLAIN BEFORE YOU CODE
`Before implementing any non-trivial function, write a`
`plain-English description (3-5 sentences) of:`
`  - What it does`
`  - What algorithm or pattern it uses, and why`
`  - Inputs, outputs, and side effects`
`  - Known edge cases and how they are handled`
`This description becomes the function's docstring.`
`If writing the description reveals a design flaw, fix it first.`
	Catches wrong design before wrong implementation — the most expensive class of error. An agent that cannot describe an approach clearly probably has not thought it through.

• LOG EVERY ASSUMPTION
`Whenever you assume something not explicitly stated (a port, a default value, an ambiguous requirement), log it immediately:`
`  ASSUMPTION [#N]: what you assumed`
`  Reason: why`
`  Impact if wrong: what breaks`
`Collect all assumptions in an ASSUMPTIONS.md file.`
`Never assume silently.`
	Assumptions are invisible bugs. Making them explicit turns silent failures into visible, auditable items you can review and correct before they cause problems downstream.

• RUN → CAPTURE → FIX → REPEAT
`After writing or modifying code, run it immediately.`
`For each error:`
`  a. State the exact error message`
`  b. Hypothesize the root cause (1-2 sentences)`
`  c. Apply the fix`
`  d. Re-run and verify`
`Repeat until the program runs end-to-end with expected output.`
`If the same error persists after 2 fix attempts:`
`  log it as DEAD END, abandon the approach, try a new strategy.`
	Structured debugging — not brute-force retry. The 2-attempt dead-end rule is critical: without it, agents will re-try the same broken approach indefinitely and thrash.

• LOG DEAD ENDS
`When an approach fails after 2 attempts, do not retry it.`
`Log it in DEAD_ENDS.md:`
`  DEAD END [#N]: approach attempted`
`  Failed because: root cause`
`  What was tried: specific attempts`
`  Next strategy: different approach to try`
`Before trying any new approach, check DEAD_ENDS.md to avoid re-attempting known failures.`
	Prevents the most common agent loop failure. The 'Next strategy' field forces strategic thinking before action. Without this, agents silently re-enter failed paths.

• REGRESSION CHECK AFTER EVERY FIX
`After applying any fix:`
`1. List components/behaviors that were working before the fix.`
`2. Re-test each one.`
`3. Confirm explicitly before moving on:`
`  'Fix applied. Previously working: [X, Y, Z] still correct.'`
`If a regression is found, fix it before marking the bug as resolved.`
`Never close a bug without a regression check.`
	Prevents fix-one-break-two, which is the dominant cause of agent thrashing on coding tasks. A fix is not done until the regression check passes.

• MILESTONE CHECKPOINT AFTER EACH WORKING STATE
`After each phase where the code is in a working state, write`
`a MILESTONE_N.md file:`
`  Status: WORKING`
`  What works: [confirmed behaviors]`
`  What is pending: [remaining work]`
`  Decisions since last milestone: [key choices made]`
`  Assumptions since last milestone: [any new ones]`
`Do not proceed until the current milestone is confirmed working.`
	Creates a rollback ladder. If a later change breaks things, you have a known-good state to return to. Also keeps the agent's working state explicit and auditable at each stage.

• SELF-REVIEW PASS AFTER EACH MAJOR SECTION
`After completing each major component, perform a dedicated`
`review pass — separate from generation. Look for:`
`  - Logic errors, off-by-one errors, wrong assumptions`
`  - Unhandled edge cases or missing states`
`  - Inconsistencies with previous sections`
	Separating review from generation catches errors that inline writing misses. This is one of the highest-leverage instructions — it works for every task type, not just code.

```
  - Gaps between what was asked and what was
produced
Fix all issues found before continuing.
Final full-output review before delivery using the
VERIFICATION checklist.
```

- **CONTEXT CHECKPOINT IF APPROACHING TOKEN LIMIT**

```
If you are approaching your context limit or
starting a new
major phase, write CHECKPOINT.md before continuing:
  Task summary: [one sentence]
  Completed: [what is done and working]
  In progress: [current state]
  Pending: [what remains]
  Decisions made: [key choices and reasoning]
  Assumptions: [all logged assumptions]
  Known issues: [open problems]
  Next action: [exact first step when resuming]
After writing the checkpoint, you may clear working
context.
Always read CHECKPOINT.md before resuming.
```

LLM context is not degrading memory — it is a fixed window that gets truncated. This ensures nothing is lost when a long task approaches that limit. The 'Decisions made' section is most valuable: it prevents re-litigating resolved questions.

- **UPDATE DOCUMENTATION AS YOU GO**

```
Whenever you implement something that affects the
documented
behavior, update the relevant documentation
immediately.
Do not defer documentation to the end.
Specifically:
  - Update README.md when setup, usage, or behavior
changes
  - Update docstrings when function contracts change
  - Update PLAN.md when architecture decisions
change
  - Update ASSUMPTIONS.md when new assumptions are
made
Documentation is out of date the moment
implementation diverges from it.
```

Documentation written at the end is almost always incomplete or inaccurate. Updating in-place keeps docs trustworthy and ensures you always have an accurate reference while implementing.

Minimum set for any coding task: Plan First · Explain Before You Code · Run→Capture→Fix→Repeat · Regression Check · Self-Review Pass.

For longer or multi-session tasks, add: Log Every Assumption · Log Dead Ends · Milestone Checkpoint · Context Checkpoint.

Part II — Skeleton Section Reference

> *This section is for you — the human reading this guide. None of this text goes into your prompt. It explains what to put in each skeleton section and why.*

ROLE

Sets the cognitive lens. An agent framed as a 'senior security engineer' weights security trade-offs differently than one framed as a 'junior Python developer'. The role shapes every downstream decision.

	Example
✅**Strong**	You are a senior Python engineer specializing in FastAPI and async systems, known for writing clean, production-grade, well-tested code with thorough inline documentation.
❌**Weak**	You are an AI assistant. Help me with my code.

CONTEXT

Eliminates assumptions — the single largest cause of off-target outputs. The agent cannot see your codebase, your org constraints, or your previous decisions. Context bridges every one of those gaps.

- **Situation:** what already exists, what triggered the task, what has been tried and failed
- **Domain:** tech stack, language version, frameworks, hard architectural constraints
- **Audience:** who uses the output, their expertise level, their expectations

Rule of thumb: if removing a sentence could cause the agent to make a wrong assumption, keep it.

TASK

Defines the work precisely. One clear primary objective plus numbered sub-tasks consistently outperforms a wall of loosely described requirements.

- Primary objective: one sentence, action verb first — write / build / refactor / analyze / plan
- Sub-tasks: numbered breakdown for multi-component work
- Explicit scope: state what is out of scope as clearly as what is in scope

CONSTRAINTS

Shapes the solution space. Without constraints, agents optimize for a reasonable generic default. Explicit constraints direct effort toward your actual requirements.

- **Must Do:** non-negotiable requirements (security, performance targets, licensing, style guide)
- **Must Not Do:** hard exclusions ('Do not use jQuery' beats 'use modern code')
- **Quality Bar:** production-ready | peer-review level | internal draft | proof-of-concept

OUTPUT FORMAT

Specifies exactly how the result is delivered. Unspecified format produces over-explained, under-structured, or inconsistently styled output — even when the underlying content is correct.

- **Structure:** sections, code blocks, tables, ordering
- **Style:** formal/informal, technical depth, active/passive voice
- **Length:** target word count, lines of code, number of sections

EXAMPLES

Calibrates quality and style faster than any written description. A 10-line positive example is worth 100 words of instruction. A negative example prevents the single most predictable failure mode.

- Always include when tone or voice is critical
- Always include when the output must match a specific structural pattern
- Skip when the task is purely computational with an unambiguous output format

VERIFICATION

An explicit self-review checklist. Writing it as task-specific assertions (not generic reminders) is what makes it work.

	Example check
✗ **Generic**	Check that your output is good quality.
✓ **Specific**	Every function has a docstring. No TODOs remain. No library is imported that was not in the dependencies list. The code runs without modification from a clean environment.

Part III — Full Prompt Examples

Five complete prompts, fully filled in. Each includes the skeleton sections and a tailored Runtime Commands selection.

Example 1 / 5 — Coding: REST API Endpoint

EXAMPLE: POST /tasks Endpoint — Python / FastAPI

ROLE	You are a senior Python engineer specializing in FastAPI and async systems. You write clean, type-annotated, production-ready code with thorough inline documentation.
CONTEXT Situation	Building a task management API. DB layer: SQLAlchemy 2.0 + PostgreSQL. Auth: JWT in headers. Need the endpoint that creates a new task.
CONTEXT Domain	Python 3.11, FastAPI 0.110, Pydantic v2. Repository pattern. Explicit session management — no ORM shortcuts.
CONTEXT Audience	Senior engineer code review → production deploy. Must be review-ready, not a prototype.
TASK	Write a POST /tasks endpoint that: 1. Accepts JSON: title (str, required), description (str, optional), due date (date, optional) 2. Validates input with a Pydantic v2 schema 3. Saves via TaskRepository (assume it exists — only call it) 4. Returns the created task, HTTP 201 5. Returns HTTP 422 for validation errors, HTTP 500 for DB errors
CONSTRAINTS	Must Do: async/await throughout, full type annotations, docstring on endpoint, try/except for DB Must Not Do: do not write TaskRepository; no test code; no global state Quality Bar: production-ready
OUTPUT FORMAT	1. Pydantic request + response schemas 2. Endpoint function 3. 3-5 line comment block at top: integration assumptions
VERIFICATION	[ ] All functions async [ ] Pydantic v2 syntax (model_config, not class Config) [ ] DB exception caught → HTTP 500 [ ] No undefined imports
RUNTIME COMMANDS	Apply: PLAN FIRST · EXPLAIN BEFORE YOU CODE · RUN→CAPTURE→FIX→REPEAT LOG EVERY ASSUMPTION · REGRESSION CHECK AFTER EVERY FIX SELF-REVIEW PASS AFTER EACH MAJOR SECTION

Example 2 / 5 — Coding: Refactor for Testability

EXAMPLE: Refactor Legacy Module — Decouple & Make Testable

ROLE	You are a senior software engineer specializing in Python refactoring and clean architecture. You apply SOLID principles pragmatically — no over-engineering.
CONTEXT Situation	I have a legacy Python module (orders.py) that mixes DB access, business logic, and email sending in the same functions. It has 0% test coverage and cannot be unit tested as written. I need to refactor it to be testable without rewriting the business logic.
CONTEXT Domain	Python 3.11. Existing tests use pytest. No DI framework available — constructor injection only. Cannot change the public function signatures (they are called by external code).
CONTEXT Audience	Team of 4 engineers. Will review via PR. Must be mergeable in one sprint.
TASK	Refactor orders.py to: 1. Separate DB access, business logic, and email sending into distinct layers 2. Inject dependencies via constructors so they can be mocked in tests 3. Keep all existing public function signatures unchanged 4. Write unit tests for the business logic layer using pytest + unittest.mock
CONSTRAINTS	Must Do: public signatures unchanged; constructor injection only; pytest for tests Must Not Do: no new frameworks; no changes to DB schema; do not merge layers Quality Bar: all new code ≥ 80% test coverage
OUTPUT FORMAT	1. Refactored module structure (file list with responsibilities) 2. Refactored orders.py 3. Test file: test_orders.py 4. Migration note: what callers need to update (if anything)
VERIFICATION	[ ] All public signatures identical to originals [ ] No function touches more than one layer [ ] Tests mock all external dependencies (DB, email) [ ] pytest runs clean with no failures
RUNTIME COMMANDS	Apply: PLAN FIRST · EXPLAIN BEFORE YOU CODE · LOG EVERY ASSUMPTION RUN→CAPTURE→FIX→REPEAT · REGRESSION CHECK AFTER EVERY FIX SELF-REVIEW PASS AFTER EACH MAJOR SECTION MILESTONE CHECKPOINT AFTER EACH WORKING STATE

Example 3 / 5 — Writing: Product Blog Post

EXAMPLE: Product Announcement — B2B SaaS Blog Post

ROLE	You are a senior B2B SaaS content writer specializing in product marketing. Direct, benefit-focused, zero hype. You write for decision-makers who skim.
CONTEXT Situation	Launching Flowo's biggest feature in 18 months: AI-powered anomaly detection that flags workflow bottlenecks in real time.
CONTEXT Domain	Brand voice: confident, practical, no fluff. Blog + LinkedIn cross-post. Reading level: ~10th grade.
CONTEXT Audience	Ops managers and directors at mid-market companies (100-1000 employees). Care about time savings and risk reduction, not technical mechanisms.
TASK	Write a 600-800 word blog post: 1. Hook that names the pain point

	2. What the feature does (user terms only, no ML explanation) 3. 2-3 concrete use cases 4. CTA to book a demo
CONSTRAINTS	Must Do: active voice, subheading every 150-200 words, benefits before features, second person (you/your) Must Not Do: no 'revolutionary' 'game-changing' 'cutting-edge' 'leverage'; max 800 words Quality Bar: publish-ready, no editing required
OUTPUT FORMAT	Markdown. H2 subheadings. No bullet lists in body copy.
EXAMPLES (good) **EXAMPLES (avoid)**	'Your team spent three hours last Tuesday tracking a bottleneck an alert could have caught in 30 seconds.' — names pain, quantifies, signals relief. 'We are excited to announce our groundbreaking new AI feature...' — no excitement-first, feature-first, or hype-first openings.
VERIFICATION	[ ] 600-800 words [ ] No banned words [ ] Every section opens with a benefit [ ] CTA in final paragraph
RUNTIME COMMANDS	Apply: SELF-REVIEW PASS AFTER EACH MAJOR SECTION · UPDATE DOCUMENTATION AS YOU GO

Example 4 / 5 — Planning: 90-Day Onboarding Plan

EXAMPLE: 90-Day Onboarding Plan — Senior Product Manager	
ROLE	You are a Head of People Operations with 15 years designing onboarding programs at fast-growing tech companies. You balance thoroughness with practicality.
CONTEXT Situation **CONTEXT Domain**	Onboarding a new Senior PM in two weeks. No formal plan exists. Role owns the core B2B product. First initiative: major UX overhaul due Q3. 80-person remote-first company. Slack + Notion + Jira. PM reports to CPO. Stakeholders: 2 eng leads, design lead, 3 AEs, 2 CSMs.
CONTEXT Audience **TASK** **CONSTRAINTS**	CPO (hiring manager) and the new hire. Must be self-navigable without constant hand-holding. Create a structured 90-day plan in three phases: 1. Days 1-30: Orient (product, team, processes, customers) 2. Days 31-60: Contribute (first ownership, relationship-building) 3. Days 61-90: Lead (own a roadmap slice, run planning cycle) Must Do: specific activities per phase, measurable success criteria, flag 3-4 onboarding risks Must Not Do: no generic HR boilerplate (IT setup, benefits); not so dense it overwhelms Quality Bar: hand-off ready, no editing required
OUTPUT FORMAT	Table per phase: Week | Focus Area | Key Activities | Success Criteria Follow with bullet-list 'Common Pitfalls' section. ~600-800 words excluding tables.
VERIFICATION	[ ] Ownership expectations escalate clearly phase by phase [ ] All success criteria are measurable (no 'understands the product') [ ] Pitfalls are PM-role-specific, not generic onboarding

RUNTIME COMMANDS	Apply: LOG EVERY ASSUMPTION · SELF-REVIEW PASS AFTER EACH MAJOR SECTION

Example 5 / 5 — Analysis: Competitive Positioning

EXAMPLE: Competitive Positioning Gap Analysis	
ROLE	You are a strategic analyst specializing in B2B SaaS competitive intelligence. You synthesize positioning data into specific, actionable recommendations grounded in evidence.
CONTEXT Situation	Head of Product at Teamly — project management for engineering teams (10-50 devs, mid-market). Competitor data from public sites + G2 reviews: Linear: loved for speed/minimalism; weak reporting Asana: strong cross-team; bloated for engineering Jira: deep dev integration; complex, slow onboarding
CONTEXT Domain	Our edge: fastest bug→shipped-fix path, deep GitHub/GitLab integration. Known weakness: analytics, stakeholder reporting.
TASK	Produce: 1. Positioning gap map — uncontested spaces we could own 2. Top 3 exploitable competitor weaknesses (one each) 3. Top 2 vulnerabilities vs. each competitor 4. Recommended positioning statement from identified gaps
CONSTRAINTS	Must Do: ground every claim in provided data; flag inferences explicitly; specific not directional recs Must Not Do: do not address our analytics weakness here; no generic SWOT Quality Bar: board-presentation ready
OUTPUT FORMAT	1. Gap Map: narrative + 2x2 table (X=complexity, Y=dev-team fit) 2. Exploitable Weaknesses: 3 bullets with rationale 3. Vulnerability Table: competitor vs. our exposure 4. Positioning Statement: 1-2 sentences ~500-700 words
VERIFICATION	[ ] Every weakness claim traces to provided data [ ] Positioning statement excludes all three competitors specifically [ ] All inferences are flagged as such
RUNTIME COMMANDS	Apply: LOG EVERY ASSUMPTION · SELF-REVIEW PASS AFTER EACH MAJOR SECTION